Thuật toán Tham lam (Greedy Algorithm)

Lập trình C++ Nâng cao

Tấn Đức

July 1, 2026

Nội dung chính

1. Câu chuyện dẫn nhập
2. Khái niệm Tham lam
3. Bài toán Đổi tiền (Coin Change)
4. Bài toán Lập lịch (Activity Selection)
5. Bài toán Cái túi phân số (Fractional Knapsack)
6. Kinh nghiệm và Tổng kết

Câu chuyện thực tế: Đi siêu thị

Tình huống

Bạn mua đồ hết $15k$ và đưa cho thu ngân tờ $100k$. Thu ngân cần thối lại $85k$.

Thu ngân sẽ lấy các tờ tiền nào để **số lượng tờ tiền đưa cho bạn là ít nhất**?

Cách thu ngân làm (Bản năng):

- Lấy tờ lớn nhất có thể: **50k** (còn $35k$).
- Lấy tờ lớn nhất tiếp theo: **20k** (còn $15k$).
- Lấy tờ lớn nhất tiếp theo: **10k** (còn $5k$).
- Lấy tờ lớn nhất cuối cùng: **5k** (đủ $85k$).

Nhận xét

Việc luôn ưu tiên chọn mệnh giá **lớn nhất ở thời điểm hiện tại** mà không cần nhìn xa hơn chính là bản chất của **Thuật toán Tham lam (Greedy)**.

Thuật toán Tham lam là gì?

Định nghĩa

Thuật toán tham lam (Greedy Algorithm) là phương pháp giải bài toán tối ưu bằng cách luôn đưa ra **lựa chọn tốt nhất tại thời điểm hiện tại** (tối ưu cục bộ - local optimum), với hy vọng chuỗi các lựa chọn này sẽ dẫn đến kết quả tốt nhất cho toàn bộ bài toán (tối ưu toàn cục - global optimum).

Đặc điểm

- **Tốc độ:** Thường rất nhanh, độ phức tạp đa số phụ thuộc vào bước sắp xếp, thường là $\mathcal{O}(N \log N)$ hoặc $\mathcal{O}(N)$.
- **Bộ nhớ:** Không tốn kém bộ nhớ, $\mathcal{O}(1)$ hoặc $\mathcal{O}(N)$.
- **Thiết kế:** Dễ code, tư duy trực quan.

Bài toán 1: Đổi tiền (Coin Change)

Đề bài

Cho hệ thống tiền tệ của Việt Nam gồm các mệnh giá: $C = \{1, 2, 5, 10, 20, 50, 100, 200, 500\}$ (đơn vị: nghìn đồng).
Bạn cần đổi số tiền S bằng cách sử dụng **ít tờ tiền nhất**. Giả sử số lượng tờ tiền mỗi mệnh giá là vô hạn.
Input: $S = 285$
Output: 5 (Gồm các tờ: 200, 50, 20, 10, 5)

Ý tưởng thuật toán

1. Sắp xếp các mệnh giá giảm dần: $500 \to 200 \to \cdots \to 1$.
2. Duyệt qua từng mệnh giá C_i:
3. Lấy tối đa số tờ C_i có thể: $cnt = S/C_i$.
4. Cập nhật lại $S = S \pmod{C_i}$.

Cài đặt bài toán Đổi tiền

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Mang luu cac menh gia giam dan
    vector<int> coins = {500, 200, 100, 50, 20, 10, 5, 2, 1};
    int S = 285;
    int ans = 0;

    for (int c : coins) {
        ans += S / c; // Lay toi da so to menh gia c
        S %= c;       // Phan tien con lai
    }

    cout << "So to tien it nhat: " << ans << endl;
    return 0;
}
```

Khi nào Tham lam thất bại?

Sai lầm thường gặp

Thuật toán tham lam **KHÔNG** phải lúc nào cũng đúng cho mọi hệ thống tiền tệ. Nó chỉ đúng khi hệ thống tiền tệ mang tính chất "chuẩn" (canonical coin systems).

Phản ví dụ:

- Hệ thống tiền: $C = \{1, 3, 4\}$
- Số tiền cần đổi: $S = 6$

So sánh

Tham lam: $6 = 4 + 1 + 1$ (3 tờ)
Tối ưu thực sự: $6 = 3 + 3$ (2 tờ)
$\Longrightarrow$ Tham lam cho kết quả sai!

Khắc phục: Với hệ thống tiền tệ bất kỳ, ta phải dùng Quy hoạch động (Dynamic Programming).

Bài toán 2: Lập lịch (Activity Selection)

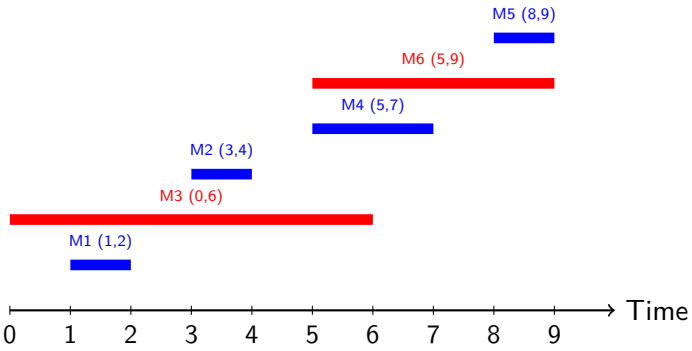
Đề bài

Có N cuộc họp, cuộc họp thứ i bắt đầu tại thời điểm S_i và kết thúc tại thời điểm E_i. Một phòng họp chỉ có thể tổ chức một cuộc họp tại một thời điểm. Hãy tìm **số lượng cuộc họp tối đa** có thể được tổ chức.

Cuộc họp	1	2	3	4	5	6
Bắt đầu (S_i)	1	3	0	5	8	5
Kết thúc (E_i)	2	4	6	7	9	9

Chiến lược tối ưu

Ưu tiên chọn các cuộc họp **kết thúc sớm nhất**. Bởi vì cuộc họp kết thúc càng sớm thì khoảng thời gian trống còn lại trong ngày càng nhiều để tổ chức các cuộc họp khác.



Cài đặt bài toán Lập lịch

```cpp
// Cau truc luu tru thoi gian
struct Meeting { int start, end; };
// Ham so sanh de sort theo thoi gian ket thuc (E_i) tang dan
bool cmp(Meeting a, Meeting b) { return a.end < b.end; }
int main() {
    vector<Meeting> a = {{1,2}, {3,4}, {0,6}, {5,7}, {8,9}, {5,9}};
    sort(a.begin(), a.end(), cmp);

    int cnt = 0, last_end = -1;
    for (auto m : a) {
        if (m.start >= last_end) { // Hop le khong chong cheo
            cnt++;
            last_end = m.end;
        }
    }
    cout << "So cuoc hop toi da: " << cnt << endl;
    return 0;
}
```

Bài toán 3: Vắt sữa

Tóm tắt đề bài

Một đàn bò gồm n con, con thứ i dự kiến vắt được a_i lít sữa. Đặc tính của đàn bò là: Cứ mỗi lần vắt sữa một con, những con bò **còn lại chưa được vắt** sẽ sợ và bị giảm sản lượng đi đúng 1 lít sữa (nếu sản lượng ≤ 0 thì lượng sữa vắt được là 0).

Yêu cầu: Hãy xác định thứ tự vắt sữa để tổng số lít sữa thu được là **nhiều nhất**.

vatsua.inp	**vatsua.out**	**vatsua.inp**	**vatsua.out**
4 4 4 4 4	10	4 2 1 4 3	6

Giải quyết bài toán Vắt sữa

Nhận xét

Nếu ta vắt sữa một con bò ở lượt thứ k (tính từ lượt 0), sản lượng của nó sẽ bị trừ đi đúng k lít. Do đó, lượng sữa thu được từ con bò này sẽ là: $\max(0, a_i - k)$.

Ý tưởng Tham lam (Greedy)

Để lượng sữa hao hụt ảnh hưởng ít nhất đến tổng thu hoạch, ta cần ưu tiên vắt những con bò có **sản lượng ban đầu lớn nhất** trước.

1. Sắp xếp mảng sản lượng A theo thứ tự **giảm dần**.
2. Lần lượt duyệt từng con bò ở vị trí i (từ 0 đến $n-1$).
3. Cộng dồn $\max(0, A[i] - i)$ vào tổng. Nếu phát hiện $A[i] - i \leq 0$, ta có thể **dừng thuật toán sớm** vì các con bò phía sau chắc chắn cũng sẽ cho 0 lít.

Code mẫu: Vắt sữa

Lưu ý (Edge Case)

Tổng lượng sữa tối đa có thể đạt $n \times \max(a_i) = 10^5 \times 10^9 = 10^{14}$. Con số này vượt quá giới hạn của số nguyên 32-bit, do đó bắt buộc phải sử dụng `long long` cho biến lưu tổng kết quả!

```
void solve(vector<int> &a, int n){
        sort(a.begin(), a.end(), greater<long long>());
        long long max_milk = 0;
        for (int i = 0; i < n; ++i) {
            if (a[i] - i <= 0) break; // Dung som neu san luong da can kiet
            max_milk += (a[i] - i);
        }

        cout << max_milk << "\n";
}
```

Kinh nghiệm thi đấu (Competitive Programming)

Mẹo thi đấu CP

- **Dấu hiệu nhận biết:** Các bài toán yêu cầu tìm "lớn nhất", "nhỏ nhất", "ít nhất", "nhiều nhất" nhưng các lựa chọn độc lập và hướng tới một hàm mục tiêu đơn giản.
- **Chìa khóa của Tham lam là Sắp xếp (Sort):** Hầu như bài Greedy nào cũng yêu cầu bạn định nghĩa một hàm `cmp()` custom để `sort` dữ liệu theo một tiêu chí (ví dụ: kết thúc sớm nhất, tỷ lệ lớn nhất, ...).
- **Kiểm chứng bằng phản ví dụ:** Khi nảy ra ý tưởng Tham lam, hãy dành 10 giây nghĩ xem có test case nhỏ nào phá vỡ ý tưởng của mình không trước khi vội vàng code. Nếu có, hãy chuyển hướng sang **Quy hoạch động (DP)** hoặc **Quay lui (Backtracking)**.

Tổng kết chương

Những điểm cần ghi nhớ

1. **Bản chất:** Tối ưu cục bộ dẫn đến tối ưu toàn cục.
2. **Ưu điểm:** Code ngắn, chạy rất nhanh (chủ yếu là $O(N \log N)$ do thuật toán Sort).
3. **Nhược điểm:** Không phải bài nào cũng dùng được. Cần chứng minh tính đúng đắn ngầm trong đầu.
4. **Các bài toán kinh điển:** Đổi tiền (hệ chuẩn), Lập lịch, Cái túi phân số, Nối dây thừng (dùng Hàng đợi ưu tiên).

Cảm ơn các bạn đã theo dõi!